APRESENTACAO TECNICA

FiscalMove FMS: do diagnostico em PRs ao Motor Fiscal

Uma solucao de fretes onde a operacao fica no sistema Java, a inteligencia fiscal fica em uma API Go, e a decisao arquitetural nasce de dados reais de manutencao.

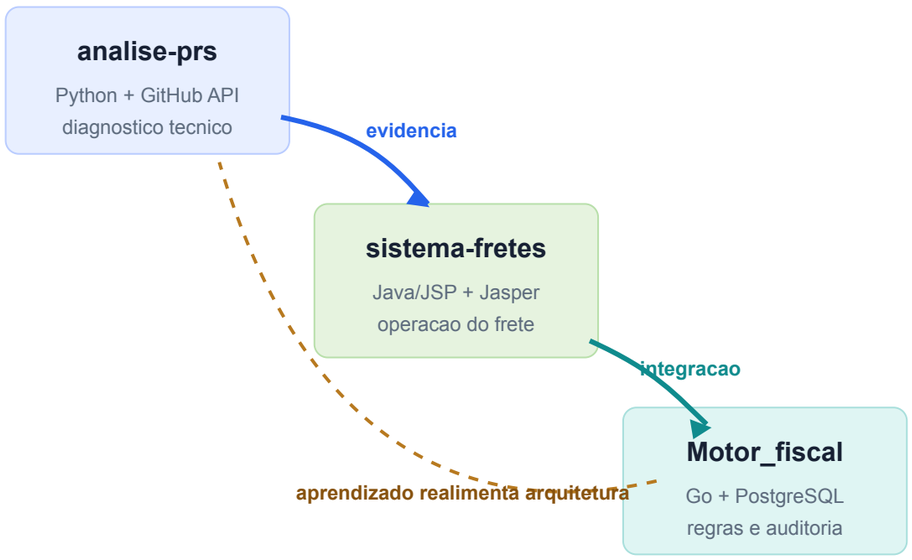
FiscalMove
Sistema de fretes
Java + Go + PostgreSQL

Sistema-de-frete >
analise-prs >
motor-fiscal-go >

Tese central

Separar regra fiscal da operacao reduziu acoplamento e tornou o calculo explicavel.

O projeto combina diagnostico, arquitetura e implementacao: primeiro foram analisados PRs, depois o sistema de fretes foi estruturado, e por fim o Motor Fiscal centralizou a decisao tributaria.



analise-prs
Python

Antes de propor arquitetura, foram analisados dados reais de PRs

O projeto Python transformou historico de manutencao em insumo tecnico para identificar dores recorrentes.

Pipeline de coleta e tratamento

GitHub API
PRs fechados, titulo, autor, data e corpo do PR.

>

extrair_prs.py
Coleta paginada, retry, classificacao e salvamento incremental.

>

limpar_dados.py
Normalizacao, score de qualidade e bases para analise.

analise_ia.csv
274 registros com maior qualidade para leitura aprofundada.

>

gerar_dashboard.py
Resumo por tipo, categoria, causa e evolucao mensal.

>

Dashboard
Base para justificar decisao arquitetural.

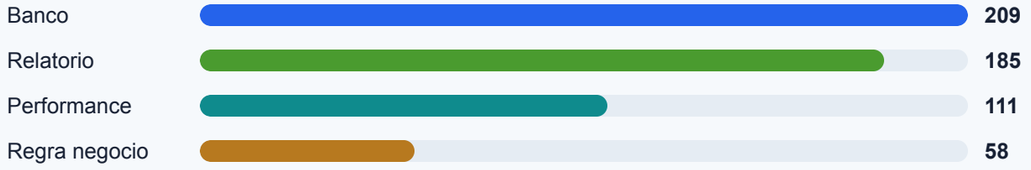
Numeros da base local

13.397
PRs coletados

2
repositorios analisados

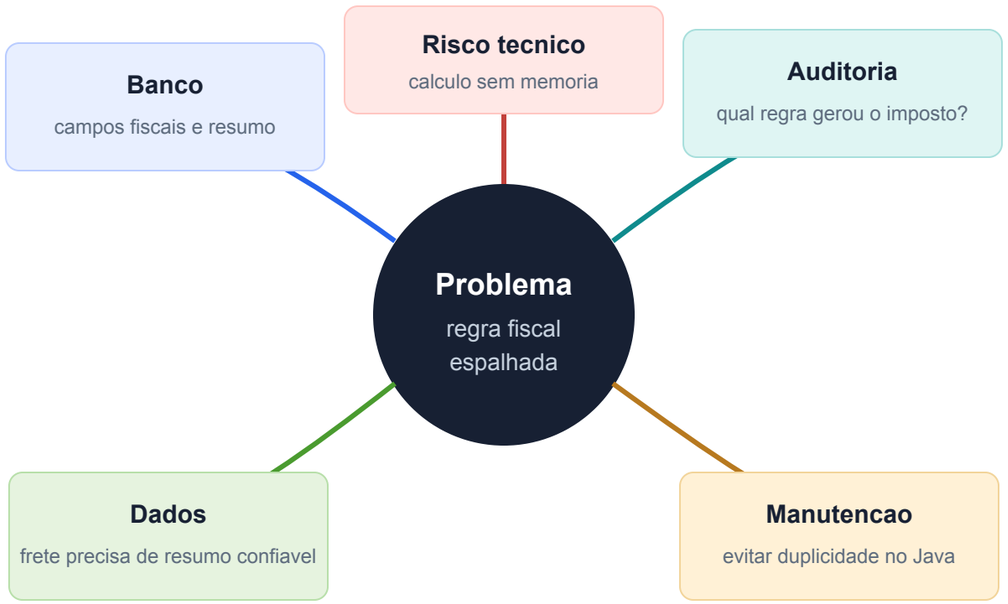
55
autores identificados

2019-2026
periodo analisado



As dores apontavam para uma regra fiscal espalhada e dificil de explicar

A leitura dos PRs e do sistema levou a uma separacao de responsabilidades mais clara.



Conclusao do diagnostico

A solucao nao era apenas criar mais uma tela.

A dor pedia uma decisao arquitetural: manter o sistema Java focado na operacao e mover a decisao fiscal para um servico especializado.

1. **Operacao**
   fretes, clientes, motoristas, veiculos e ocorrencias.
2. **Dados operacionais**
   frete, resumo fiscal e ocorrencias precisam estar consistentes.
3. **Fiscal**
   regras, calculos e auditoria concentrados no Motor Fiscal.

ARQUITETURA PENSADA

Tres projetos, tres responsabilidades bem separadas

Cada repositorio tem uma funcao clara na historia tecnica da solucao.

Arquitetura
Separacao por papel

1. Diagnostico

Python

Coleta e analisa PRs para revelar recorrencia de manutencao.

```
extrair_prs.py
limpar_dados.py
gerar_dashboard.py
```

GitHub API · CSV · dashboard · evidencia

2. Operacao

Java

Executa o fluxo operacional de fretes e consome o resultado fiscal da API.

```
FreteBO
RelatorioBO
MotorFiscalClient
```

Servlets · JSP · Jasper · PostgreSQL

3. Fiscal

Go

Centraliza regras, impostos, simulacoes e memoria de calculo.

```
FiscalRuleService
TaxCalculator
FiscalSimulationRepository
```

API REST · rules engine · decimal · auditavel

O Java continua responsavel pelo ciclo operacional

sistema-fretes
Java/JSP

O sistema principal mantem cadastro, validacao, emissao, status, ocorrencias e resumo fiscal retornado pela API.

1

Cadastros base

Clientes, motoristas e veiculos com validacoes de CPF/CNPJ, CNH, status e capacidade.

BO + DAO

2

Emissao do frete

Valida datas, peso, valores, origem, destino, motorista ativo e veiculo disponivel.

FreteBO

3

Calculo fiscal

Java monta o payload e chama o Motor Fiscal com API Key e Correlation ID.

HTTP

4

Status e ocorrencias

Controla a maquina de estados: emitido, saida, transito, entregue, nao entregue e cancelado.

Workflow

5

Consolidacao

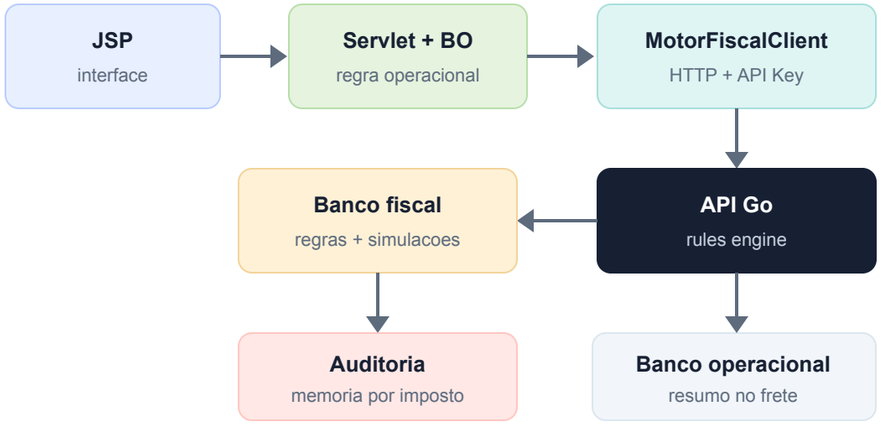
Resumo fiscal, ocorrencias e documentos ficam disponiveis para consulta operacional.

Dados

O desenho final separa operacao, regra fiscal e auditoria

A arquitetura foi pensada para reduzir acoplamento, proteger credenciais, manter contrato claro e permitir evolucao fiscal sem espalhar regra no Java.

Fluxo arquitetural escolhido



Por que essa arquitetura?

ESCOLHA	RESULTADO TECNICO
Java como sistema principal	Mantem telas, cadastros, fretes, status e documentos operacionais.
Go como servico fiscal	Isola regra tributaria em uma API menor, testavel e evolutiva.
Dois bancos conceituais	Operacao e auditoria fiscal nao ficam misturadas.
Contrato HTTP	Integra sem acoplar classes internas entre Java e Go.
API Key no backend	Token nao fica exposto na tela ou no JavaScript.
Correlation ID	A mesma operacao pode ser rastreada entre Java, Go e banco.

A API Go virou o nucleo tecnico da decisao fiscal

Ela concentra contrato, seguranca, motor de regras, calculo com decimal, persistencia e memoria de calculo.

Motor_fiscal
Go + PostgreSQL

Contrato e responsabilidades

API
HTTP

- /health verifica disponibilidade.
- /tax/preview calcula antes de salvar o frete.
- /tax/simulate calcula e persiste auditoria.

Service
Rules

- TaxService valida e orquestra o fluxo.
- FiscalRuleService aplica matching, prioridade e conflito.
- TaxCalculator calcula ICMS, IBS e CBS com decimal.
- Erros retornam JSON padronizado para o Java tratar.

Dados
SQL

- fiscal_rules guarda vigencia, CFOP, prioridade e status.
- fiscal_rule_conditions decide quando a regra vale.
- fiscal_rule_taxes guarda aliquotas por imposto.
- fiscal_simulations registra memoria fiscal.

Por que Go aqui?

1. **Servico pequeno e focado**
   A API faz uma coisa bem definida: decisao fiscal.
2. **Contrato independente**
   O Java nao precisa conhecer classes internas do motor.
3. **Calculo previsivel**
   Valores trafegam como string e sao calculados com `decimal`, nao `float`.
4. **Regra versionavel**
   Vigencia, status, prioridade, fonte e revisao contabil ficam modelados.
5. **Auditoria nativa**
   Cada simulacao guarda regra aplicada, impostos e formula de calculo.

Escolha: contrato HTTP, regra no banco, calculo no service, auditoria persistida

INTEGRACAO FINALIZADA

O Java chama o Go, recebe o calculo e grava o resumo no frete

Contrato
HTTP + JSON

A integracao foi pensada para nao expor token no navegador e para permitir rastreabilidade entre sistemas.

sistema-fretes

Java

- Tela de frete solicita preview ou emissao.
- `FreteBO` valida dados operacionais.
- `TaxSimulationRequest.fromFrete` monta payload.
- `MotorFiscalClient` envia JSON com headers.
- `FreteDAO.atualizarResumoFiscal` grava retorno.

Motor Fiscal

Go

- Recebe `POST /api/v1/tax/simulate`.
- Valida `X-API-Key`.
- Propaga `X-Correlation-ID`.
- Seleciona regra fiscal aplicavel.
- Retorna CFOP, impostos, regra e memoria.

PostgreSQL

Dados

- Sistema Java: tabela `frete` guarda resumo fiscal.
- Motor Fiscal: `fiscal_rules` guarda regras.
- Motor Fiscal: `fiscal_simulations` guarda historico.
- Motor Fiscal: `fiscal_simulation_tax_details` guarda memoria por imposto.

Seguranca: token fica no backend; rastreabilidade: correlation ID

RESULTADO TECNICO

Arquitetura fiscal orientada por dados e preparada para evolucao

A solucao organiza diagnostico, operacao e decisao fiscal em camadas independentes, com contrato claro e auditoria persistida.

Resultado da composicao

13.397
PRs usados como base de diagnostico tecnico

3
repositorios com responsabilidades separadas

API
contrato fiscal independente do sistema Java

SQL
regras, simulacoes e memoria de calculo auditaveis

analise-prs · Sistema-de-frete · Motor_fiscal · OpenAPI · PostgreSQL

Valor tecnico entregue

1. Decisao arquitetural sustentada por diagnostico de manutencao.
2. Separacao entre operacao de frete e decisao fiscal.
3. API Go com regras no banco, prioridade, vigencia, status e auditoria.
4. Integracao segura com API Key e rastreavel por Correlation ID.
5. Memoria de calculo persistida por simulacao e por imposto.